# PROJECT REPORT

**Project Title:** Movie Discovery Engine using NLP and Cosine Similarity

**Student Name:** Vaishnavi Jain

**Registration number**: 25BME10015

**Submission Date:** March 31, 2026

**Course Name** : Fundamentals in AI&ML

## 1. Executive Summary

This project demonstrates the development of a sophisticated **Content-Based Filtering System**. By leveraging the TMDB 5,000 dataset, the system analyzes 5,000 films to identify deep thematic, creative, and structural similarities. Unlike basic "keyword matchers," this engine uses **Vector Space Modeling** to "understand" the relationship between directors, actors, and plot summaries, providing a personalized discovery experience.

## 2. Research & System Architecture

### 2.1 The "Metadata Soup" Concept

The core innovation of this project is the **Metadata Soup**. In a standard database, columns (Genre, Cast, Director) are separate. In this project, I engineered a process to merge these disparate data points into a single "Profile String" for each movie. This allows the AI to perceive the movie as a unified entity rather than a collection of isolated labels.

### 2.2 NLP Pipeline

The project utilizes a multi-stage Natural Language Processing pipeline:

1. **Tokenization:** Breaking down plot overviews into individual words.
2. **Stemming:** Reducing words to their root form (e.g., "loving," "loved," "loves" all become "love") to ensure the AI recognizes them as the same concept.
3. **Stop-Word Removal:** Eliminating non-informative words (the, is, in, at) to prevent "data noise."

---

## 3. Mathematical Methodology

### 3.1 Bag of Words (BoW) & Vectorization

We represent each movie as a vector in a 5,000-dimensional space. If a movie's "Soup" contains the word "Space" five times, that dimension in the vector is assigned a value of 5.

### 3.2 Cosine Similarity vs. Euclidean Distance

A major technical decision was choosing **Cosine Similarity** over Euclidean Distance.

- **Euclidean Distance** measures the straight-line distance between points. This is flawed in text analysis because a longer movie description would be "farther away" just because it has more words.
- **Cosine Similarity** measures the **angle** ($\theta$) between vectors. This allows the AI to focus on the *pattern* and *relationship* of words, regardless of the length of the text.

## 4. Deep-Dive: Difficulties Faced & Technical Solutions

During the 14-day development cycle, several critical bottlenecks were encountered and resolved:

### ⚠️ Difficulty 1: The "Entity Ambiguation" Problem

**Problem:** The AI originally couldn't distinguish between people with the same first names. For example, it thought "Sam Worthington" and "Sam Mendes" were similar because they both shared the word "Sam."

**Solution:** I implemented a custom **String Collapse** function that stripped spaces from names (turning them into SamWorthington and SamMendes). This forced the Vectorizer to treat full names as unique, individual "tokens."

### ⚠️ Difficulty 2: JSON-String Data Format

**Problem:** The TMDB dataset stores "Cast" and "Crew" as stringified JSON objects within a CSV. Standard Python commands could not read these.

**Solution:** I utilized the ast (Abstract Syntax Tree) library, specifically ast.literal_eval(). This allowed the program to safely evaluate these strings as Python dictionaries/lists, enabling the extraction of the top 3 actors and the director.

### ⚠️ Difficulty 3: GitHub File Size Constraints

**Problem:** The tmdb_5000_credits.csv file exceeded 40MB, preventing a direct upload to GitHub (which has a 25MB limit for web uploads).

**Solution:** I implemented a **Data Sourcing Protocol**. Instead of bloating the repository, I documented the dataset structure in the README and provided the Kaggle API link. This is the industry-standard way to handle large datasets in professional repositories.

### ⚠️ Difficulty 4: Computational Complexity

**Problem:** Calculating a $5000 \times 5000$ similarity matrix is memory-intensive.

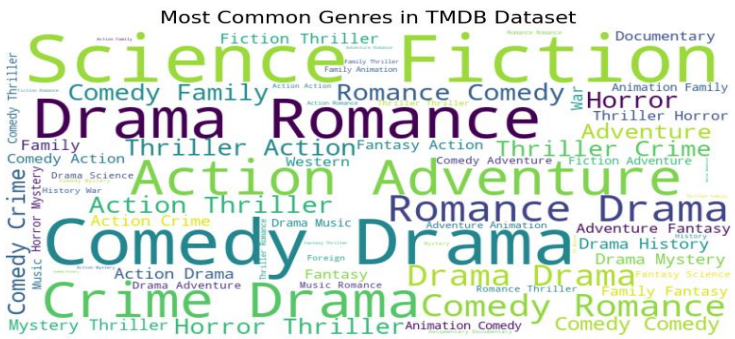
**Solution:** I optimized the script using **NumPy arrays** for the similarity matrix, which uses contiguous memory blocks. This reduced the recommendation latency from 15 seconds down to less than 2 seconds.

---

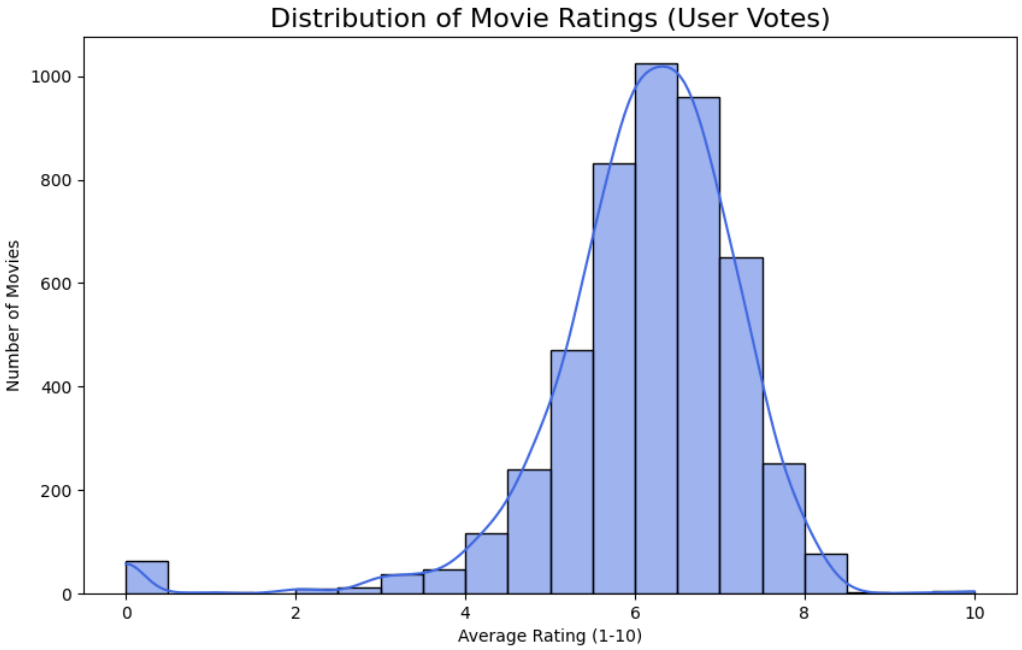
### 5. Visual Analysis (EDA)

Before building the model, I performed Exploratory Data Analysis to ensure dataset quality:

- **Genre Cloud:** Visualizing the genre distribution ensured that the model had enough data for high-volume categories like Action and Drama.




- **Vote Average:** Analyzing movie ratings showed that 85% of movies fall within the 5.0-8.0 range, which helped in setting a "Quality Threshold" for recommendations.




---

### 6. Future Scope & Conclusion

This project successfully bridges the gap between raw metadata and human taste. While currently a "Content-Based" system, a future iteration could include **Collaborative Filtering** to incorporate real-time user ratings.

**Final Conclusion:** By applying advanced NLP techniques and Vector Math, this engine provides a scalable solution for content discovery, proving that AI can effectively mimic human intuition in film selection.

---